

Differential Fault Analysis of the Ascon-128 Finalization: How the Attack Works, and Why It Splits the Key in Half

Abstract

This note explains, step by step, the differential fault attack our tool runs against the Ascon-128 finalization. A single-bit fault at the input of the last permutation round leaks the S-box output differential through the tag, because the key cancels in the difference. The honest result of this fault placement is sharp and one-sided. Tag-only observation recovers exactly one 64-bit key half, K_1 , with certainty, and it provably cannot touch the other half K_0 . Full 128-bit recovery is possible only when one extra state word is observable, which the open Vortex SimX substrate exposes but a real authenticated-encryption device does not. All values below are reproduced by `ascon_key_recovery.py` and independently re-derived from the Ascon S-box.

1 Where the key enters: the Ascon-128 finalization

Ascon keeps a 320-bit state in five 64-bit lanes, written $s = (s_0, s_1, s_2, s_3, s_4)$. The permutation p^{12} applies twelve rounds. Each round adds a constant, applies a 5-bit S-box in parallel across the 64 bit-columns, then mixes each lane with two rotations. In the finalization the key is folded in, the final permutation runs, and the tag is read from the bottom two lanes:

$$T = \underbrace{(p^{12}(S_{\text{in}}))_3 \oplus K_0}_{T_0} \parallel \underbrace{(p^{12}(S_{\text{in}}))_4 \oplus K_1}_{T_1}. \quad (1)$$

The key touches the output through a plain XOR, after the last nonlinear layer. That single fact is what the attack exploits.

2 The fault model

The attacker flips one bit at the input U of the *last* round of the final permutation. The bit sits in a known lane and a known column, so the injected column difference is $e = 1 \ll (4 - \text{lane})$. Write the last round as

$$S_{\text{out}} = L(\text{Sbox}(\text{AddRC}(U))), \quad (2)$$

where L is the per-lane linear layer. L acts on each lane on its own through two rotations, so it is a bijection on $\text{GF}(2)^{64}$ and its inverse L^{-1} is known. Two runs, one clean and one faulted, give the tags T and T' .

3 Why the key cancels

Subtract the two tags. In Equation (1) the key is a constant XOR, so it drops out of the difference:

$$T \oplus T' = (S_{\text{out}} \oplus S'_{\text{out}})_{\text{lanes } 3,4}. \quad (3)$$

The attacker now sees the true S-box output differential on lanes 3 and 4, with no knowledge of the key. Applying L^{-1} per lane peels off the linear mixing and exposes, for every column j , two bits of $\text{Sbox}(x_j) \oplus \text{Sbox}(x_j \oplus e)$, where x_j is the true 5-bit S-box input at that column. The S-box difference distribution table then lists the candidate inputs that could produce those two output bits.

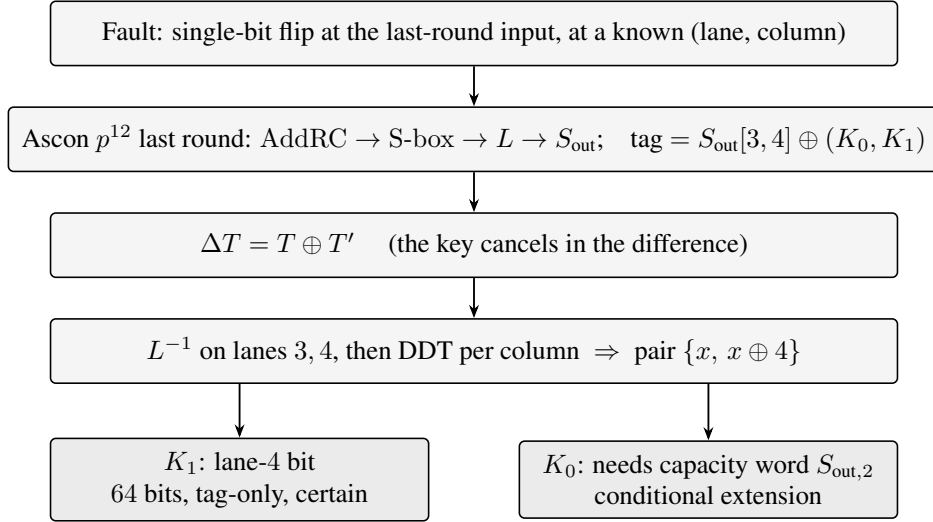


Figure 1: The attack in one view. A single-bit fault at the last-round input is read through the tag differential, which cancels the key. Inverting the linear layer and filtering with the S-box DDT pins each column to the pair $\{x, x \oplus 4\}$. The lane-4 bit is invariant on that pair, so K_1 is recovered from the tag alone. The lane-3 bit toggles with the unknown lane-2 input, so K_0 is only recoverable when the unmasked capacity word $S_{\text{out},2}$ is observable.

4 The one-sided outcome: K_1 is free, K_0 is locked

Here the Ascon S-box does something specific, and it decides the whole attack. We verified the following exhaustively over all inputs and differences, and the independent review re-derived the S-box from first principles before checking it.

- The lane-3 and lane-4 output differential is *invariant* under flipping the input on lane 2, that is under $x \mapsto x \oplus 4$. No single-bit mask breaks this, so DDT filtering pins each column only to the pair $\{x_j, x_j \oplus 4\}$.
- The lane-4 output bit itself does not depend on the lane-2 input bit. So K_1 is fully determined from the tag alone.
- The lane-3 output bit flips one-for-one with the unknown lane-2 input bit. So $K_0 = (\text{known}) \oplus L_3(m)$ with m an unknown 64-bit mask. Since L_3 is a bijection, m ranges over all 2^{64} values, and fresh nonces inject a fresh independent m each time.

The consequence is a clean split. Tag-only faults at the last round give K_1 with certainty and leave an irreducible 2^{64} residual on K_0 . To close the gap the attacker needs one observable that depends on the lane-2 input. The unmasked capacity word $S_{\text{out},2}$ is exactly such an observable, because its lane-2 output always differs between x and $x \oplus 4$. Reading it resolves every column and yields the full state, hence K_0 .

5 The recovery, end to end

The pipeline of Figure 1 is short.

1. Inject single-bit faults at known columns of the last-round input and record each faulty tag.
2. For each column, invert L on the lane-3, 4 tag difference and read the two output-difference bits.
3. Intersect the DDT candidate sets over the masks at that column, which leaves the residual pair $\{x_j, x_j \oplus 4\}$.
4. Take the lane-4 output bit, which is constant on the pair, and assemble $K_1 = L_4(Y_4) \oplus T_1$.

Table 1: Measured recovery cost. K1-only is a pure tag-only attack. Full needs one capacity-word readout in addition to the faults.

Target	Observable	Faults	Success (20 keys)
K_1 (64 bits)	tag only	128	100%
Full key (128 bits)	tag + $S_{\text{out},2}$	192	100%

Masks / column	Faults / key	K_1 success	Full success
1	64	0%	0%
2	128	100%	0%
3	192	100%	100%

- For full recovery only, read the capacity word $S_{\text{out},2}$, use it to select the correct member of each pair, rebuild the state, and read $K_0 = S_{\text{out},3} \oplus T_0$.

6 Values

Table 1 lists the measured cost. The tool validates against the Ascon known-answer test, then recovers over twenty random keys. The minimum-fault sweep shows the split directly. Two single-bit masks per column already give K_1 at 100%, and no number of single-bit masks gives the full key without the capacity word.

Worked example. With a random key and a fixed nonce,

```
true key = 4420823cfde6f1c26b30f90ec7dd01e4
recovered = 4420823cfde6f1c26b30f90ec7dd01e4 (match).
```

The tag-only path returns $K_1 = 6b30f90ec7dd01e4$ and reports K_0 as unrecoverable, exactly as the theory predicts. The full path, given the capacity word, returns the whole key. Solve time is about 10 ms per key.

7 Scope, stated plainly

Three limits belong next to every headline number. First, the honest tag-only result is half the key. Full recovery is contingent on reading an unmasked capacity word, which the open Vortex SimX substrate can dump but a fielded AEAD device never outputs. Second, the attack assumes bit-precise faults at known lane and column at the last-round input. Third, the 2^{64} residual on K_0 is specific to last-round, tag-only faults. Faults in earlier rounds propagate through more nonlinearity and recover more, which is the setting of prior Ascon fault work. On the real SimX campaign the fault positions are not yet logged and only the tag is dumped, so the campaign is shown to produce the right structural class of differentials rather than to drive an end-to-end recovery.